CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

September 18, 2025

Lecture 9

Motivation

- Many unlabeled NLP data but very few labeled data
- Can we use large amount of unlabeled data to obtain meaningful representations of words/sentences?
- meaningful representations usually mean "embedding vectors"
- human-designed features vs learned features (key focus in early deep learning era)

Learning word embeddings

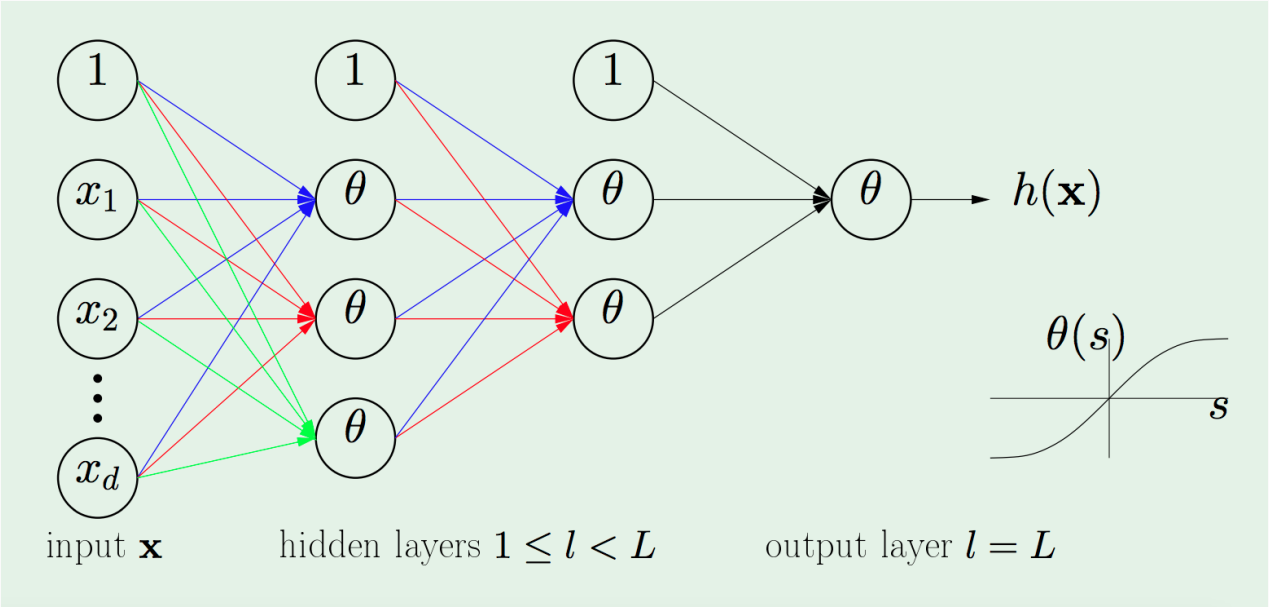
- Use large (unlabeled) corpus to learn a useful word representation
  - Learn a vector for each word based on the corpus
  - Hopefully the vector represents some semantic meaning
  - Can be used for many tasks
    - Replace the word embedding matrix for DNN models for classification/translation
- Two different perspectives but led to similar results:
  - Glove (Pennington et al., 2014)
  - Word2vec (Mikolov et al., 2013)

Neural Network

- Input layer: d neurons (input features)
- Neurons from layer 1 to L: Linear combination of previous layers + activation function

$$\theta(\boldsymbol{w}^T\boldsymbol{x}), \quad \theta: \text{ activation function}$$

- Final layer: one neuron $\Rightarrow$ prediction by sign($h(\boldsymbol{x})$)

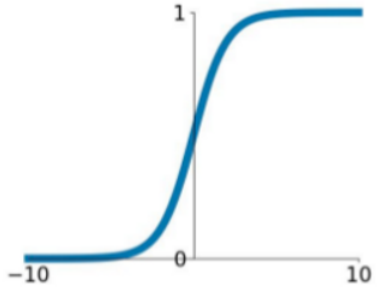


(figure from "learning from data")

Activation Function

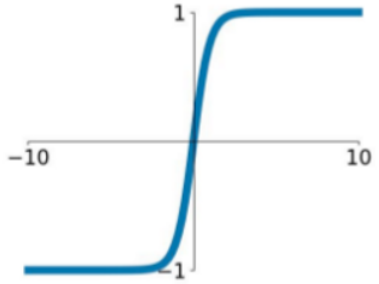
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



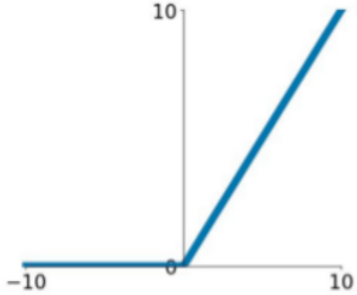
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \begin{cases} 1 \leq l \leq L & : \text{layers} \\ 0 \leq i \leq d^{(l-1)} & : \text{inputs} \\ 1 \leq j \leq d^{(l)} & : \text{outputs} \end{cases}$$

bias: $b_j^{(l)}$: added to the j-th neuron in the l-th layer

Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \begin{cases} 1 \le l \le L & : \text{layers} \\ 0 \le i \le d^{(l-1)} & : \text{inputs} \\ 1 \le j \le d^{(l)} & : \text{outputs} \end{cases}$$

bias: $b_j^{(l)}$: added to the j-th neuron in the l-th layer

j-th neuron in the l-the layer:

$$x_j^{(l)} = \theta(s_j^{(l)}) = \theta(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)})$$

Formal Definitions

$$\text{Weight: } w_{ij}^{(l)} \begin{cases} 1 \le l \le L & : \text{layers} \\ 0 \le i \le d^{(l-1)} & : \text{inputs} \\ 1 \le j \le d^{(l)} & : \text{outputs} \end{cases}$$

bias: $b_j^{(l)}$: added to the j-th neuron in the l-th layer

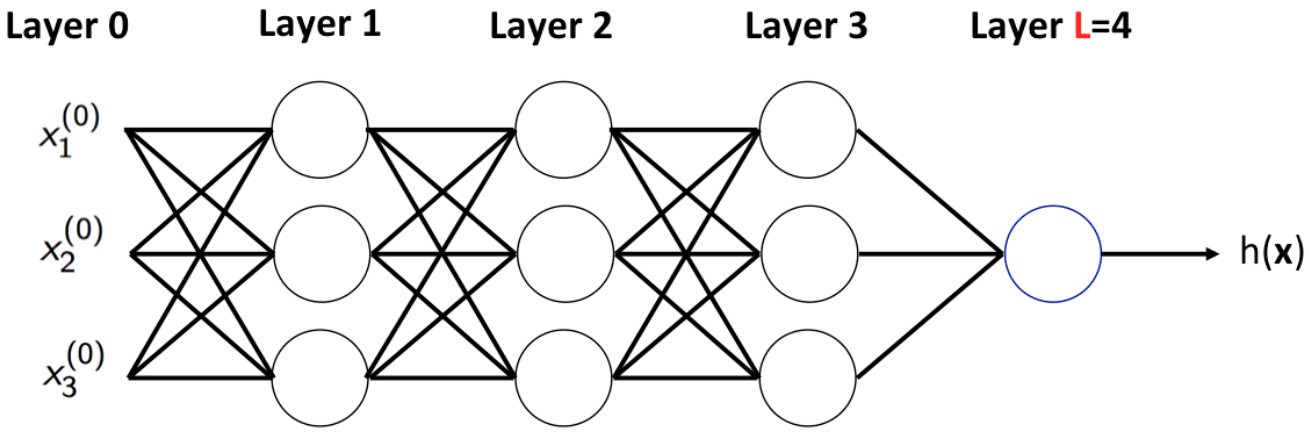
j-th neuron in the l-the layer:

$$x_j^{(l)} = \theta(s_j^{(l)}) = \theta(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} + b_j^{(l)})$$

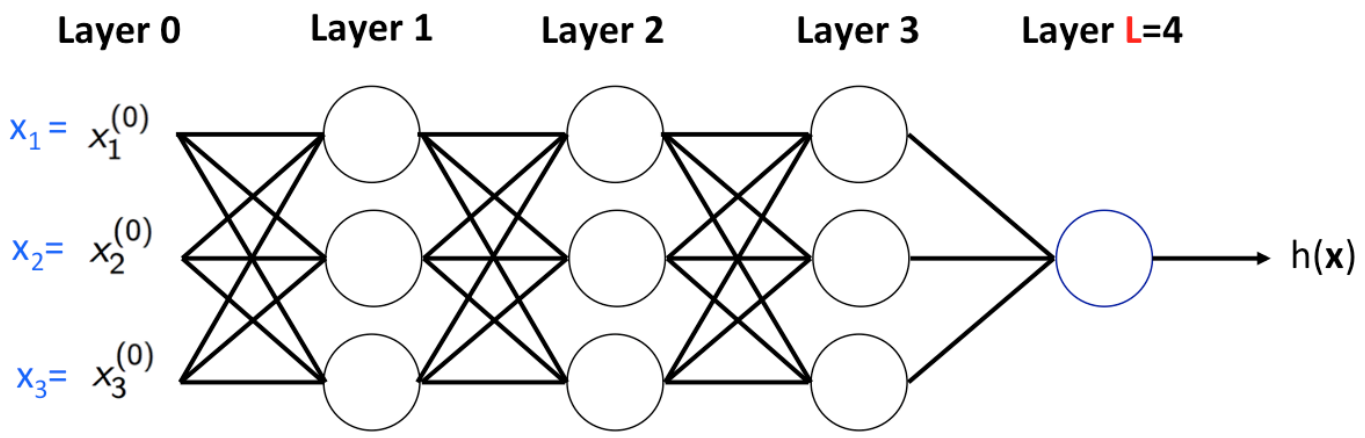
Output:

$$h(\boldsymbol{x}) = x_1^{(L)}$$

Forward propagation



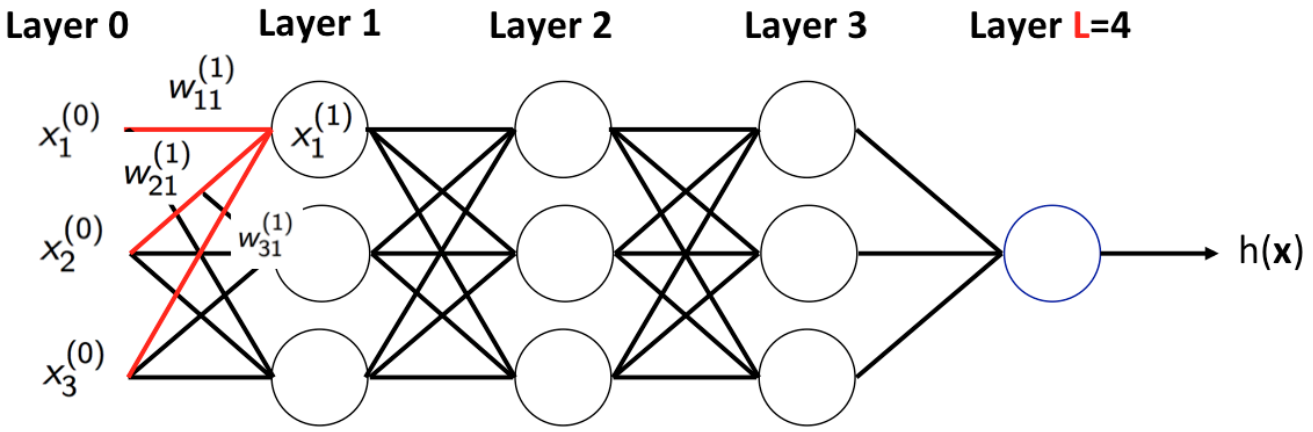
Forward propagation



features for one data point

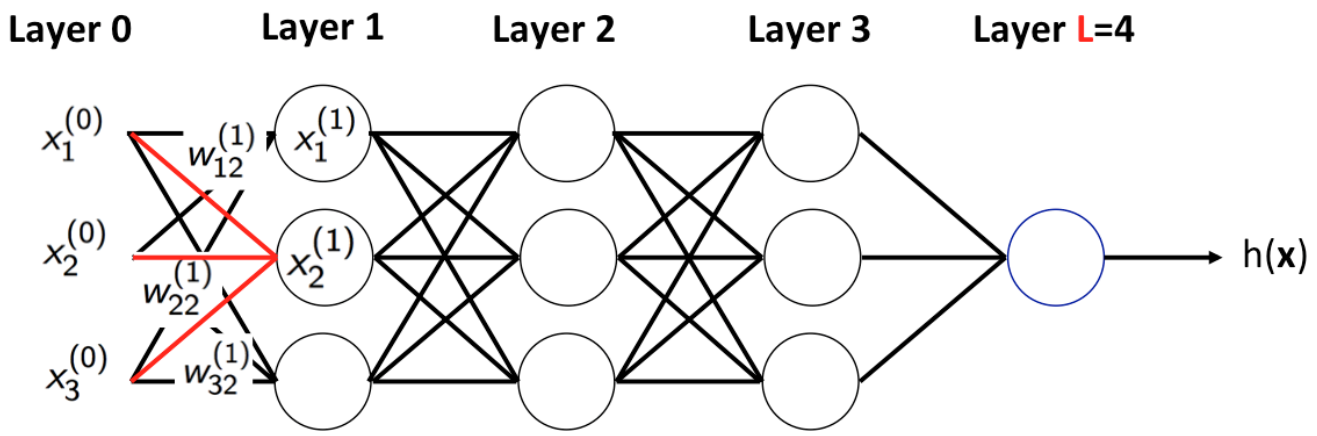
x = $[x_1, x_2, x_3]$

Forward propagation



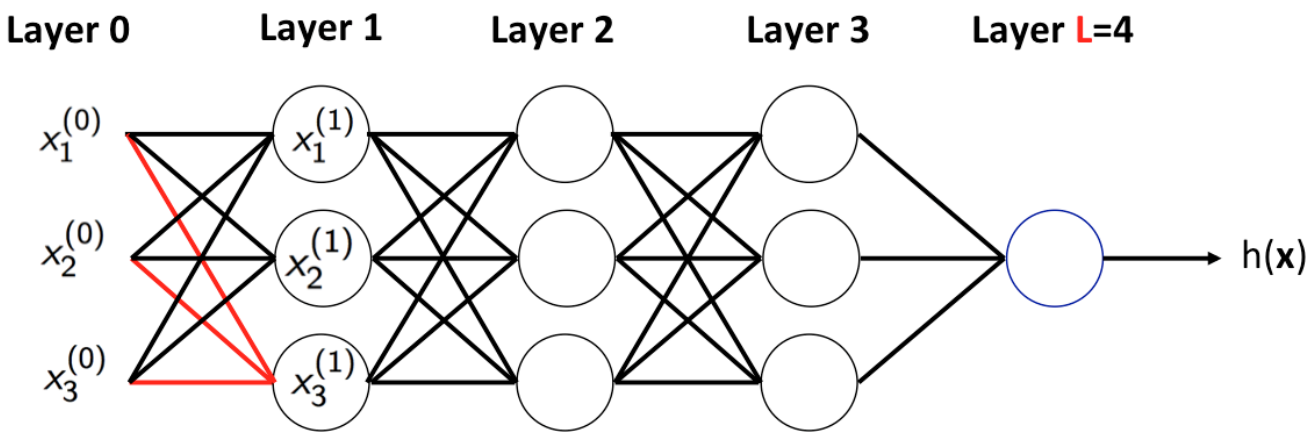
$$x_1^{(1)} = \theta\left(\sum_{i=1}^{3} w_{i1}^{(1)} x_i^{(0)}\right)$$

Forward propagation

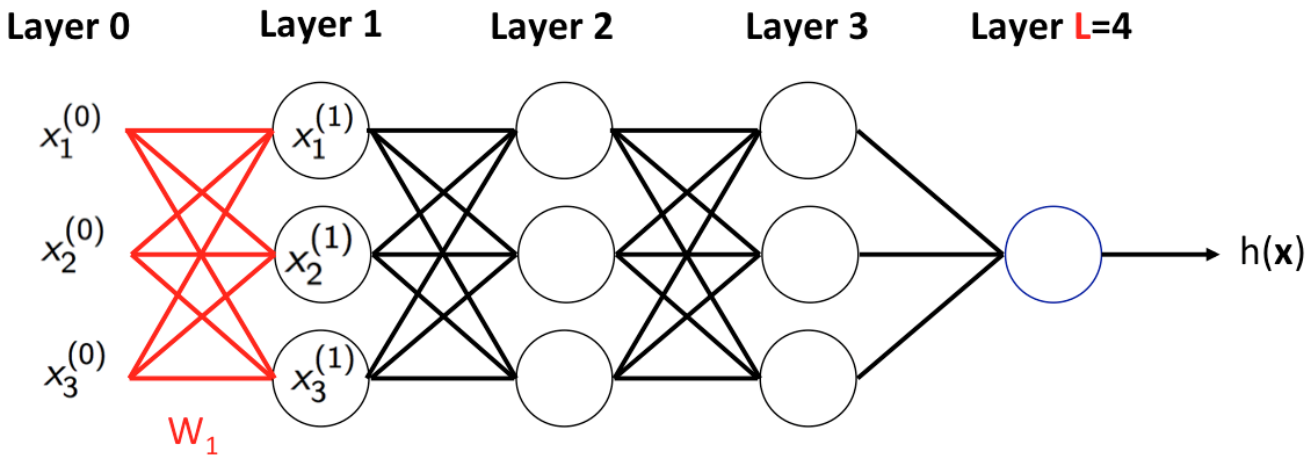


$$x_2^{(1)} = \theta\left(\sum_{i=1}^{3} w_{i2}^{(1)} x_i^{(0)}\right)$$

Forward propagation

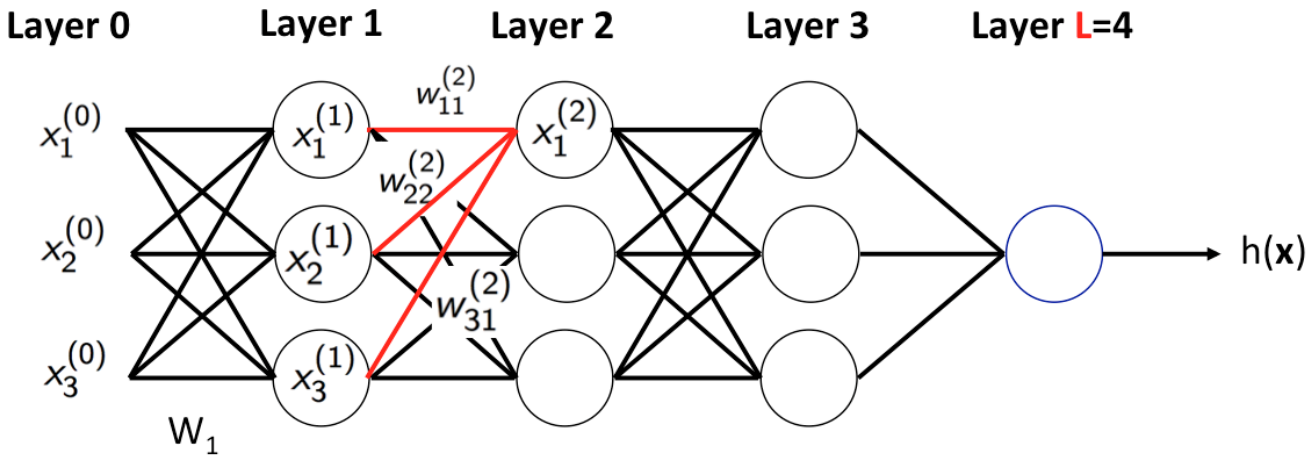


Forward propagation



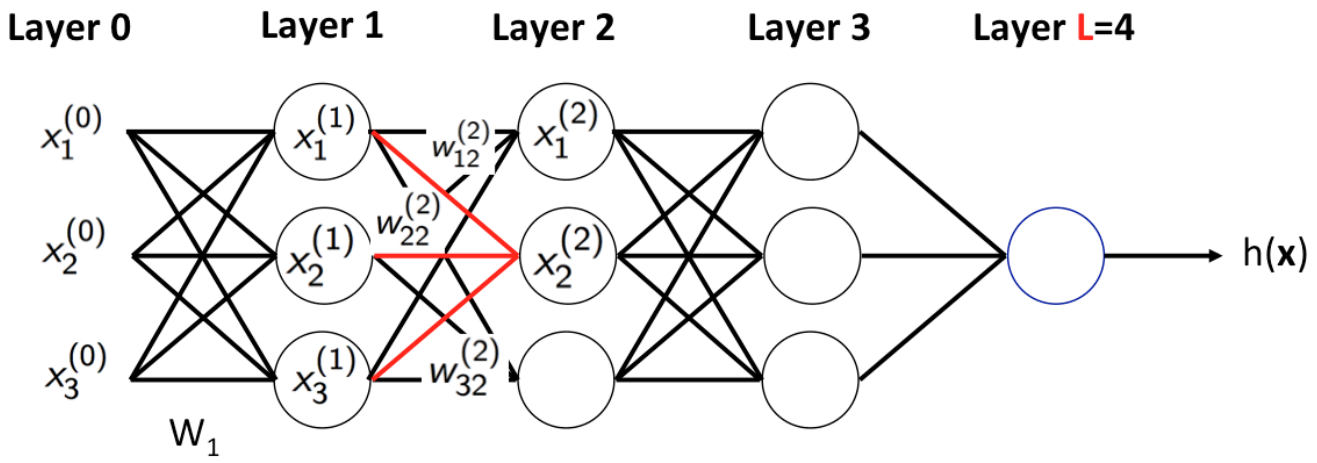
$$\boldsymbol{x}^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} = \theta \left(\begin{bmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \\ w_{13}^{(1)} & w_{23}^{(1)} & w_{33}^{(1)} \end{bmatrix} \times \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix} \right) = \theta(W_1 \boldsymbol{x}^{(0)})$$

Forward propagation



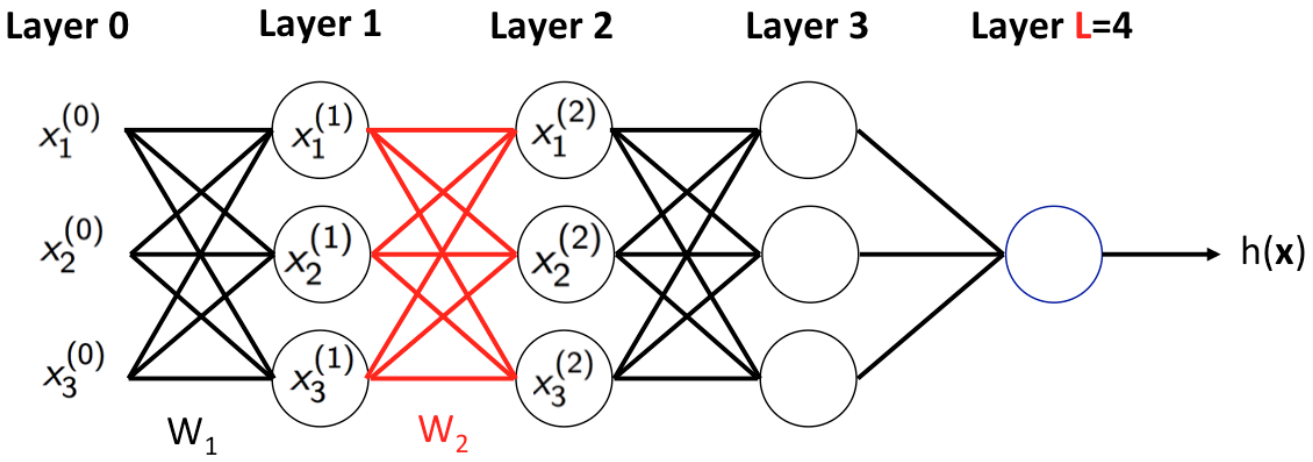
$$x_1^{(2)} = \theta\left(\sum_{i=1}^{3} w_{i1}^{(2)} x_i^{(1)}\right)$$

Forward propagation



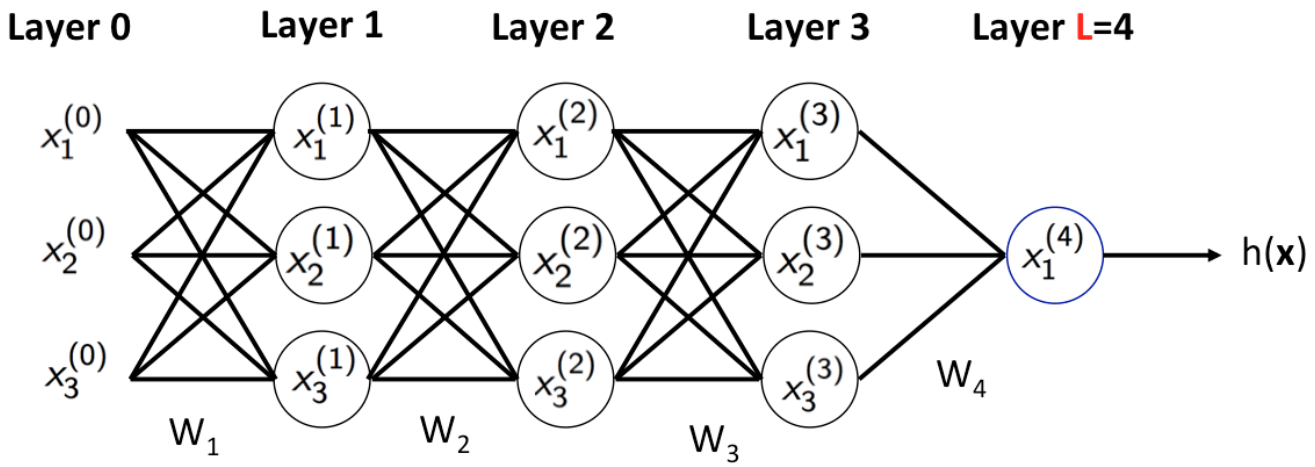
$$x_2^{(2)} = \theta\left(\sum_{i=1}^{3} w_{i2}^{(2)} x_i^{(1)}\right)$$

Forward propagation



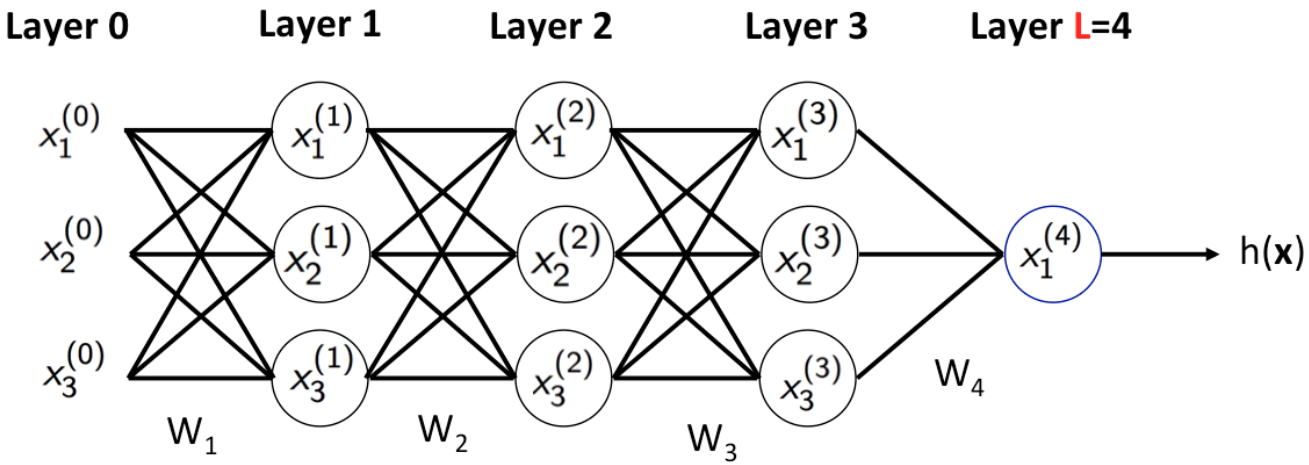
$$\boldsymbol{x}^{(2)} = \begin{bmatrix} x_1^{(2)} \\ x_2^{(2)} \\ x_3^{(2)} \end{bmatrix} = \theta \left(\begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} & w_{31}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} & w_{32}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} & w_{33}^{(2)} \end{bmatrix} \times \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} \right) = \theta(W_2 \boldsymbol{x}^{(1)})$$

Forward propagation



$$h(\boldsymbol{x}) = x_1^{(4)} = \theta(W_4 \boldsymbol{x}^{(3)}) = \theta(W_4 \theta(W_3 \boldsymbol{x}^{(2)}))$$
$$= \cdots = \theta(W_4 \theta(W_3 \theta(W_2 \theta(W_1 \boldsymbol{x}))))$$

Forward propagation



$$h(\boldsymbol{x}) = x_1^{(4)} = \theta(W_4\boldsymbol{x}^{(3)}) = \theta(W_4\theta(W_3\boldsymbol{x}^{(2)}))$$
$$= \cdots = \theta(W_4\theta(W_3\theta(W_2\theta(W_1\boldsymbol{x}))))$$

With the bias term:

$$h(\boldsymbol{x}) = \theta(W_4\theta(W_3\theta(W_2\theta(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + \boldsymbol{b}_3) + \boldsymbol{b}_4)$$

Training

- Weights $W = \{W_1, \cdots, W_L\}$ and bias $\{\boldsymbol{b}_1, \cdots, \boldsymbol{b}_L\}$ determine $h(\boldsymbol{x})$
- Learning the weights: solve ERM with SGD.
- Loss on example $(\boldsymbol{x}_n, y_n)$ is

$$e(h(\boldsymbol{x}_n), y_n) = e(W)$$

Training

- Weights $W = \{W_1, \cdots, W_L\}$ and bias $\{\boldsymbol{b}_1, \cdots, \boldsymbol{b}_L\}$ determine $h(\boldsymbol{x})$
- Learning the weights: solve ERM with SGD.
- Loss on example $(\boldsymbol{x}_n, y_n)$ is

$$e(h(\boldsymbol{x}_n), y_n) = e(W)$$

- To implement SGD, we need the gradient

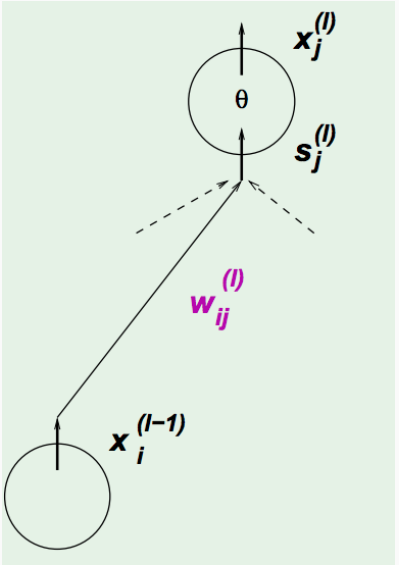
$$\nabla e(W): \quad \{\frac{\partial e(W)}{\partial w_{ij}^{(l)}}\} \text{ for all } i, j, l$$

(for simplicity we ignore bias in the derivations)

Computing Gradient $\frac{\partial e(W)}{\partial w_{ij}^{(l)}}$

- Use chain rule:

$$\frac{\partial e(W)}{\partial w_{ij}^{(l)}} = \frac{\partial e(W)}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}}$$



$s_j^{(l)} = \sum_{i=1}^{d} x_i^{(l-1)} w_{ij}^{(l)}$

- We have $\frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = x_i^{(l-1)}$

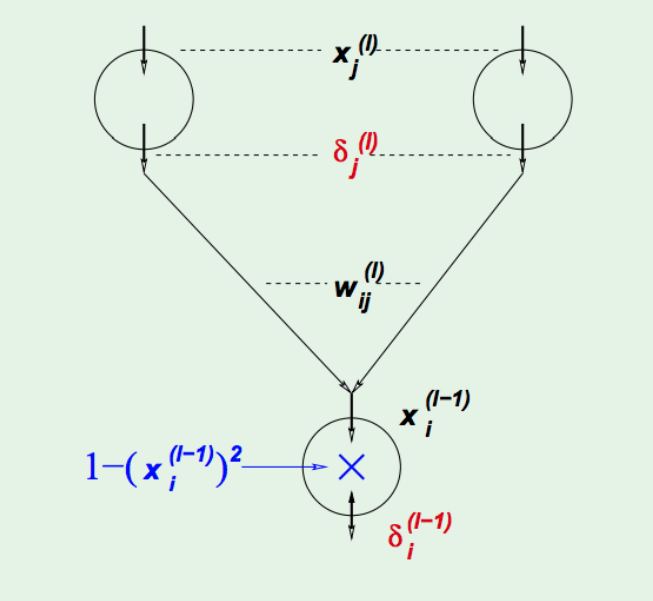
Computing Gradient $\frac{\partial e(W)}{\partial w_{ij}^{(l)}}$ (Skip)

- Define $\delta_j^{(l)} := \frac{\partial e(W)}{\partial s_j^{(l)}}$
- Compute by layer-by-layer:

$$\begin{aligned}\delta_i^{(l-1)} &= \frac{\partial e(W)}{\partial s_i^{(l-1)}} \\ &= \sum_{j=1}^{d} \frac{\partial e(W)}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial x_i^{(l-1)}} \times \frac{\partial x_i^{(l-1)}}{\partial s_i^{l-1}} \\ &= \sum_{j=1}^{d} \delta_j^{(l)} \times w_{ij}^{(l)} \times \theta'(s_i^{(l-1)}),\end{aligned}$$

where $\theta'(s) = 1 - \theta^2(s)$ for tanh

- $\delta_i^{(l-1)} = (1 - (x_i^{(l-1)})^2) \sum_{j=1}^{d} w_{ij}^{(l)} \delta_j^{(l)}$



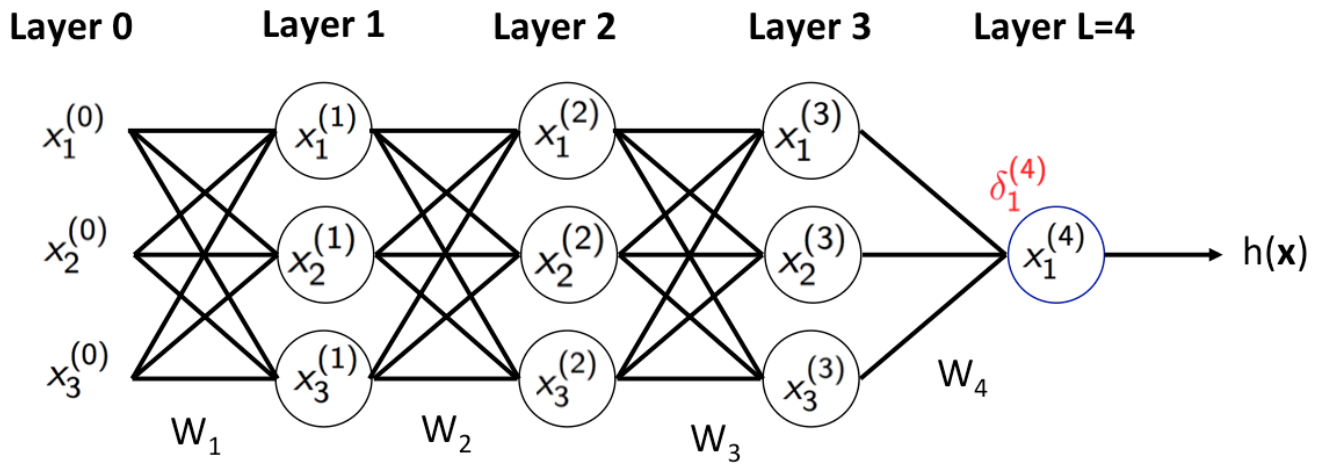
Final layer

(Assume square loss)

- $e(W) = (x_1^{(L)} - y_n)^2$
  $x_1^{(L)} = \theta(s_1^{(L)})$
- So,

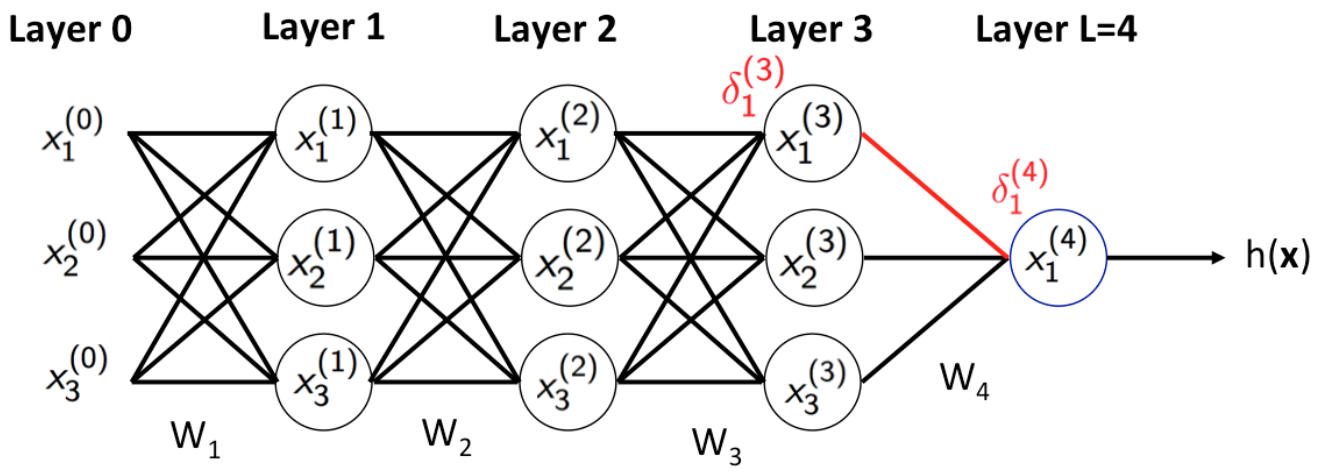
$$\begin{aligned}
\delta_1^{(L)} &= \frac{\partial e(W)}{\partial s_1^{(L)}} \\
&= \frac{\partial e(W)}{\partial x_1^{(L)}} \times \frac{\partial x_1^{(L)}}{\partial s_1^{(L)}} \\
&= 2(x_1^{(L)} - y_n) \times \theta'(s_1^{(L)})
\end{aligned}$$

Backward propagation



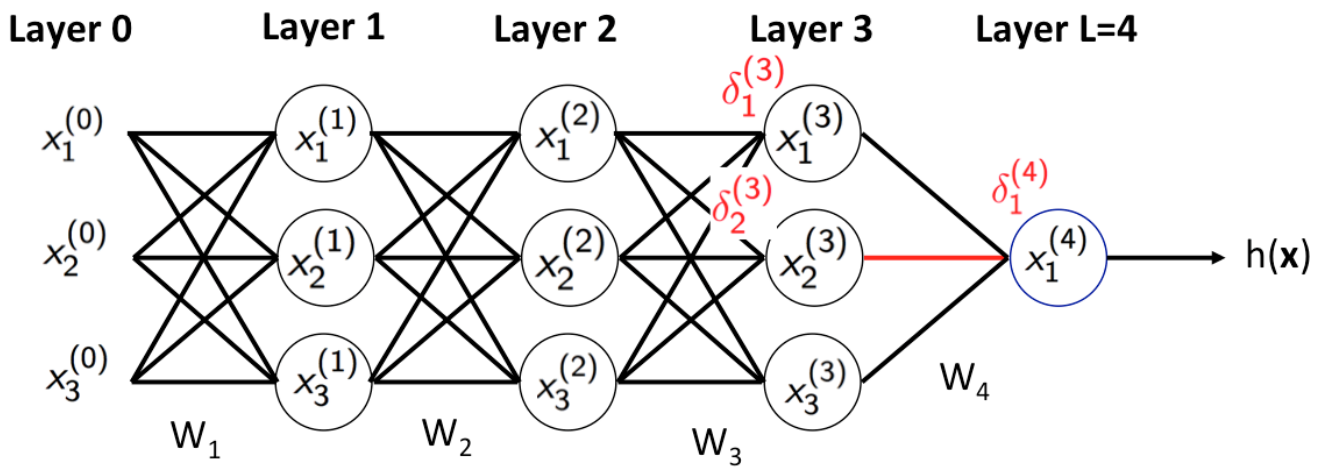
$$\delta_1^{(4)} = 2(x_1^{(4)} - y_n) \times (1 - (x_1^{(4)})^2)$$

Backward propagation



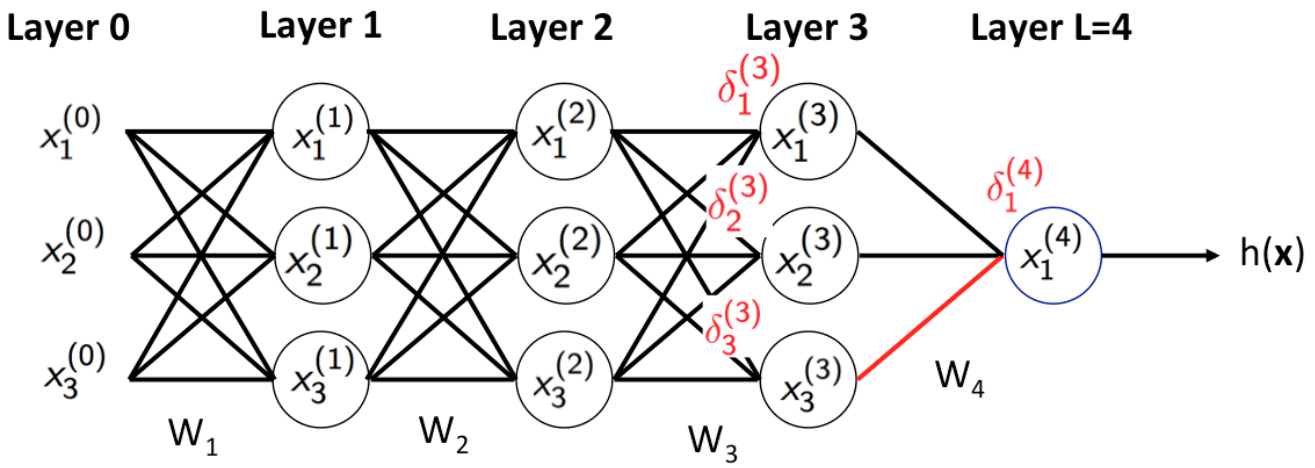
$$\delta_1^{(3)} = (1 - (x_1^{(3)})^2) \times \delta_1^{(4)} \times w_{11}^{(4)}$$

Backward propagation



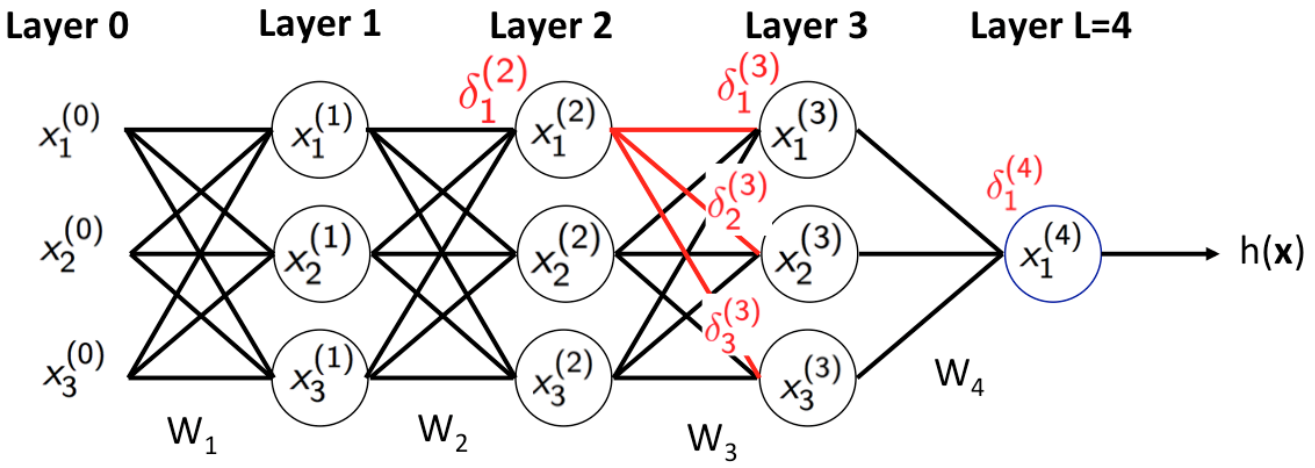
$$\delta_2^{(3)} = (1 - (x_2^{(3)})^2) \times \delta_1^{(4)} \times w_{21}^{(4)}$$

Backward propagation



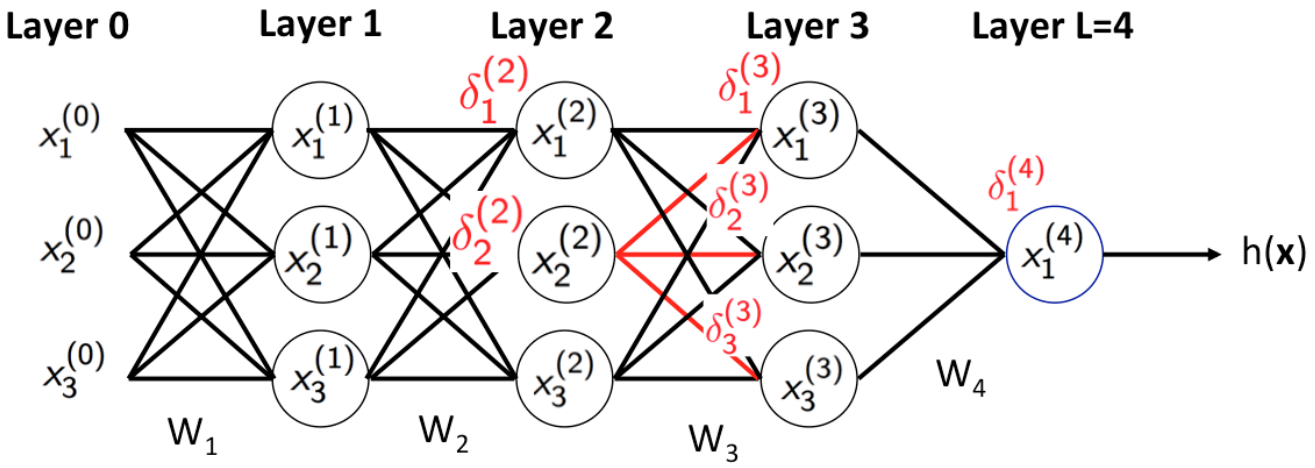
$$\delta_3^{(3)} = (1 - (x_3^{(3)})^2) \times \delta_1^{(4)} \times w_{31}^{(4)}$$

Backward propagation



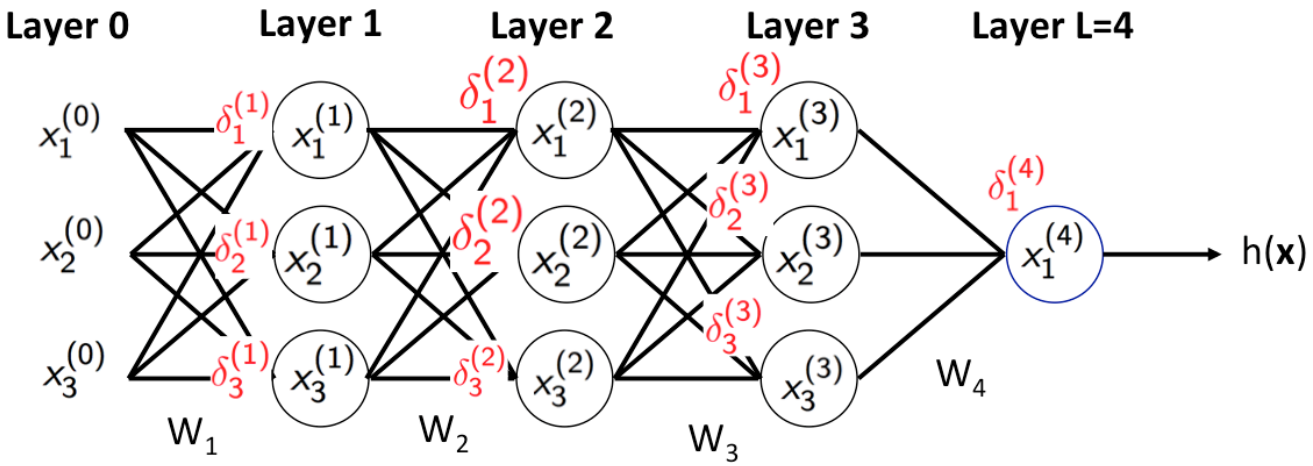
$$\delta_1^{(2)} = (1 - (x_1^{(2)})^2) \sum_{j=1}^{3} \delta_j^{(3)} w_{1j}^{(3)}$$

Backward propagation



$$\delta_2^{(2)} = (1 - (x_2^{(2)})^2) \sum_{j=1}^{3} \delta_j^{(3)} w_{2j}^{(3)}$$

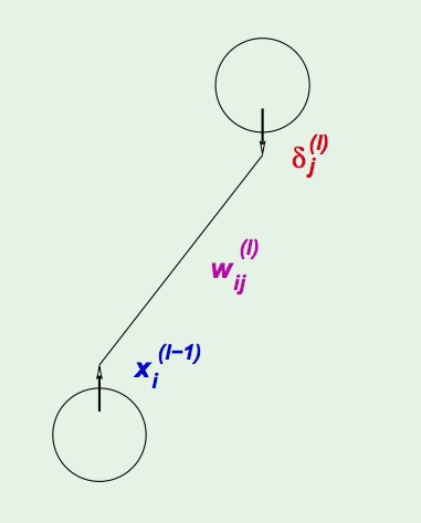
Backward propagation



Stochastic Gradient Descent (Continue Here.

SGD for neural networks

- Initialize all weights $w_{ij}^{(l)}$ **at random**
- For iter $= 0, 1, 2, \cdots$
  - Sample a data point
  - Forward: Compute all $x_j^{(l)}$ from input to output
  - Backward: Compute all $\delta_j^{(l)}$ from output to input
  - Update all the weights $w_{ij}^{l} \leftarrow w_{ij}^{(l)} - \eta x_i^{(l-1)} \delta_j^{(l)}$



Min-batch SGD

- Initialize all weights $w_{ij}^{(l)}$ **at random**
- For iter $= 0, 1, 2, \cdots$
  - Sample a batch of $|B|$ data points
  - Forward: Compute all $x_j^{(l)}$ from input to output for all samples
    (parallel matrix/tensor products)
  - Backward: Compute all $\delta_j^{(l)}$ from output to input
    (parallel matrix/tensor products)
  - Aggregate to get the average gradient
  - Update the weights

Backpropagation

- Just an automatic way to apply chain rule to compute gradient
- Auto-differentiation (AD) — as long as we define derivative for each basic function, we can use AD to compute any of their compositions
- Implemented in most deep learning packages

  (e.g., pytorch, tensorflow)

Backpropagation

- Just an automatic way to apply chain rule to compute gradient
- Auto-differentiation (AD) — as long as we define derivative for each basic function, we can use AD to compute any of their compositions
- Implemented in most deep learning packages

  (e.g., pytorch, tensorflow)
- Auto-differentiation needs to store all the intermediate nodes of each sample

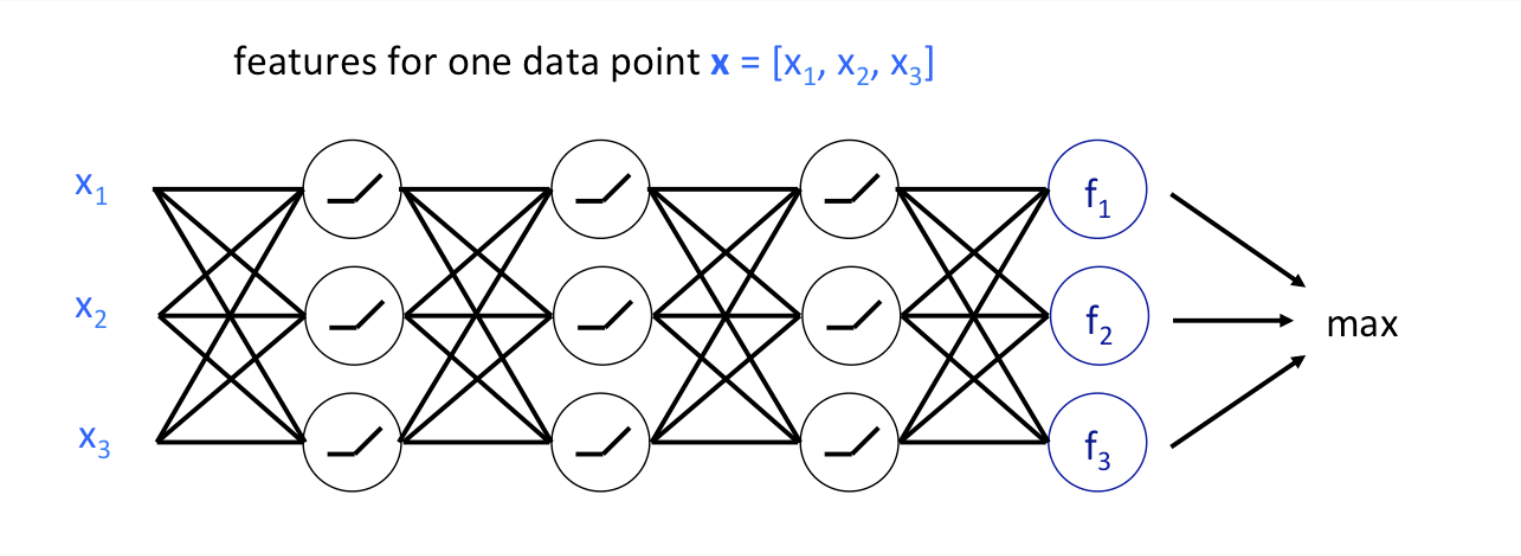
  $\Rightarrow$ Memory cost $\geq$ number of neurons $\times$ batch size

  $\Rightarrow$ This poses a constraint on the batch size

Multiclass Classification

- K classes: K neurons in the final layer
- Output of each f_i is the score of class i

  Taking $\arg\max_i f_i(x)$ as the prediction



Multiclass loss

- Softmax function: transform output to probability:

$$[f_1, \cdots, f_K] \rightarrow [p_i, \cdots, p_K]$$

  where $p_i = \frac{e^{f_i}}{\sum_{j=1}^{K} e^{f_j}}$

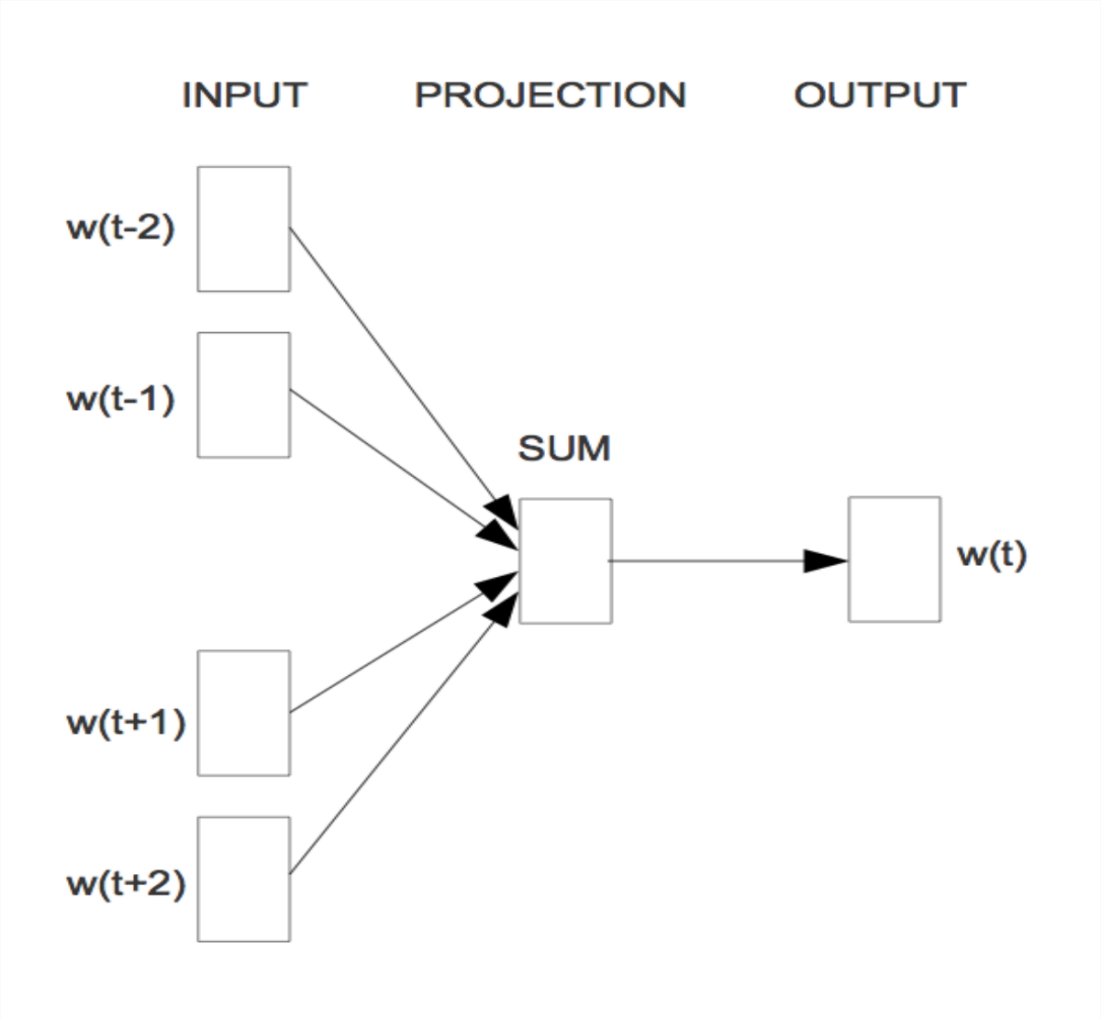
- Cross-entropy loss:

$$L = -\sum_{i=1}^{K} y_i \log(p_i)$$

  where y_i is the i-th label

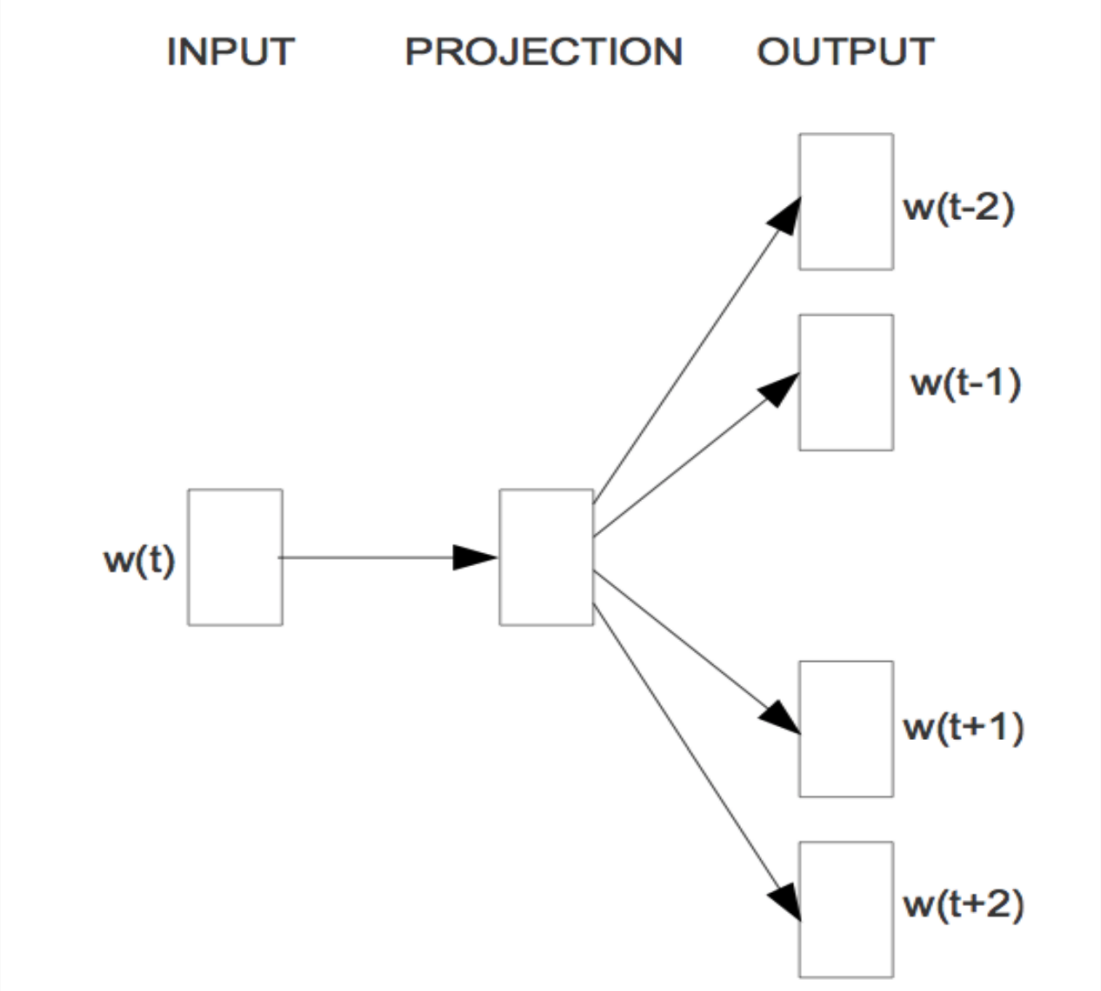
CBOW (Continuous Bag-of-Word model)

- Predict the target words based on the neighbors



Skip-gram

- Predict neighbors using target word



More on skip-gram

- Learn the probability $P(w_{t+j}|w_t)$: the probability to see w_{t+j} in target word w_t's neighborhood
- Every word has two embeddings:
  - v_i serves as the role of target
  - u_i serves as the role of context
- Model probability as softmax:

$$P(o|c) = \frac{e^{u_o^T v_c}}{\sum_{w=1}^{W} e^{u_w^T v_c}}$$

Word2vec

- Learn embeddings to minimize the cross entropy loss

$$\min_{\{u_i\},\{v_j\}} \sum_{(p,q)\in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r\neq p} e^{u_r{}^T v_q}}\right)$$

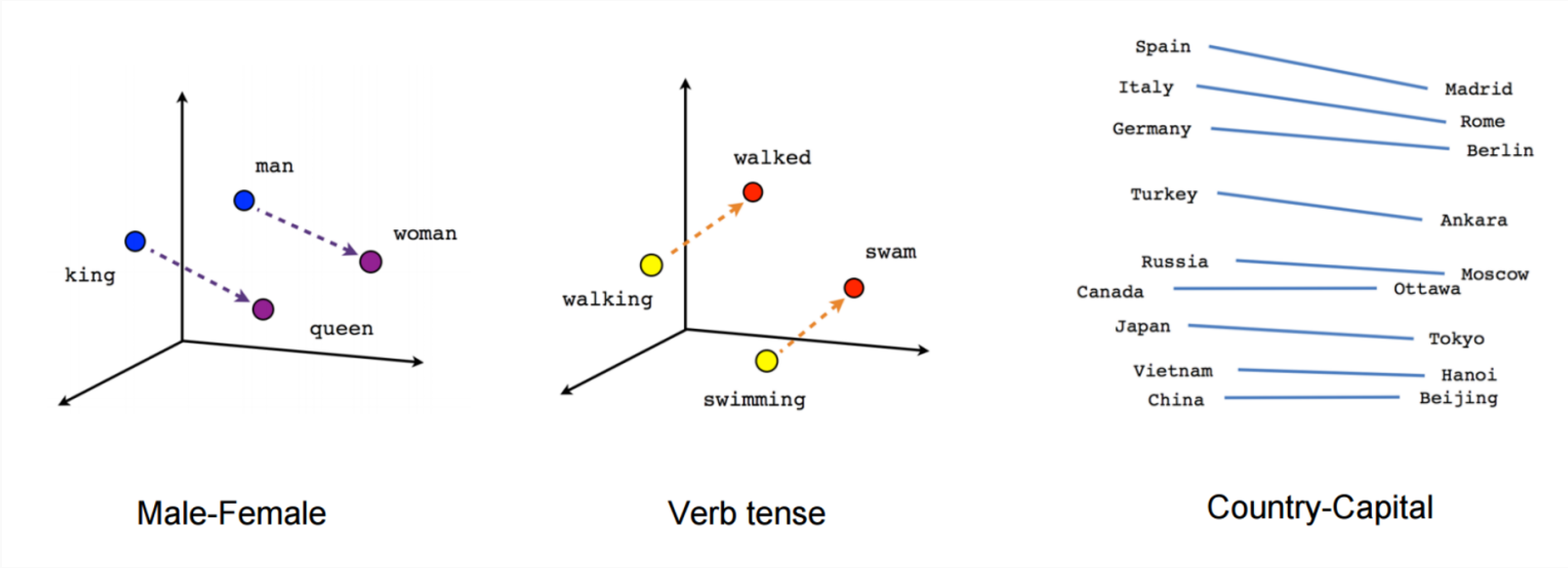
- Considering all $r \neq p$ is time consuming (size of vocabulary is large)
- Negative sampling:

$$\min_{\{u_i\},\{v_j\}} \sum_{(p,q)\in \text{positive pairs}} -\log\left(\frac{e^{u_p^T v_q}}{\sum_{r\in D_{\text{negative}}} e^{u_r{}^T v_q}}\right)$$

where D_{negative} contains a subset of negative samples.

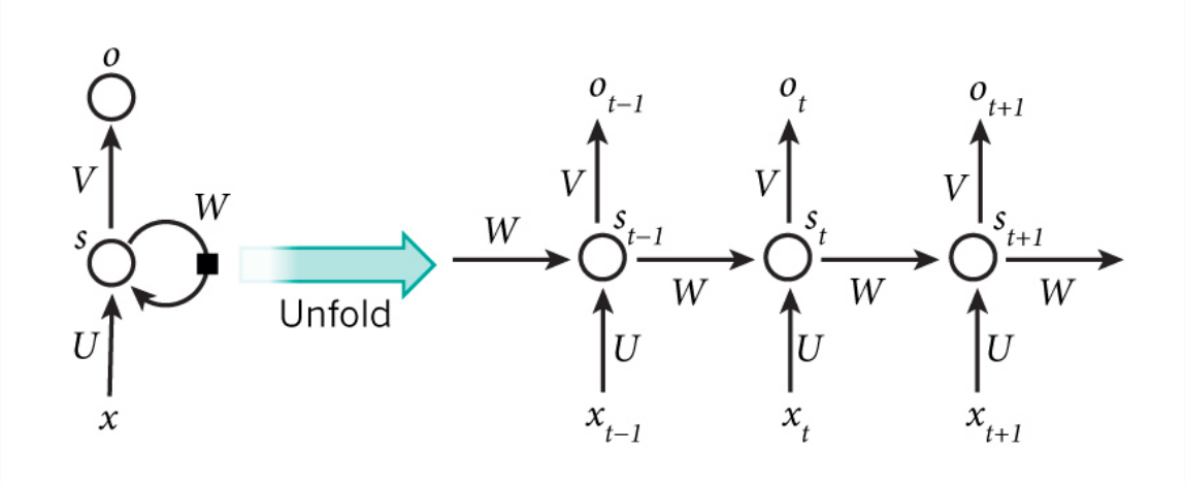
Results

The low-dimensional embeddings are (often) meaningful:



(Figure from
`https://www.tensorflow.org/tutorials/word2vec`)

Recurrent Neural Network (RNN)



- $\boldsymbol{x}_t$: t-th input
- $\boldsymbol{s}_t$: hidden state at time t ("memory" of the network)

$$\boldsymbol{s}_t = f(U\boldsymbol{x}_t + W\boldsymbol{s}_{t-1})$$

W: transition matrix, U : word embedding matrix

$\boldsymbol{s}_0$ usually set to be 0

- Predicted output at time t:

$$o_t = \arg\max_i (V\boldsymbol{s}_t)_i$$

Recurrent Neural Network (RNN)

- Training: Find U, W, V to minimize empirical loss:
- Loss of a sequence:

$$\sum_{t=1}^{T} \text{loss}(V\boldsymbol{s}_t, y_t)$$

  ($\boldsymbol{s}_t$ is a function of U, W, V)
- Loss on the whole dataset:

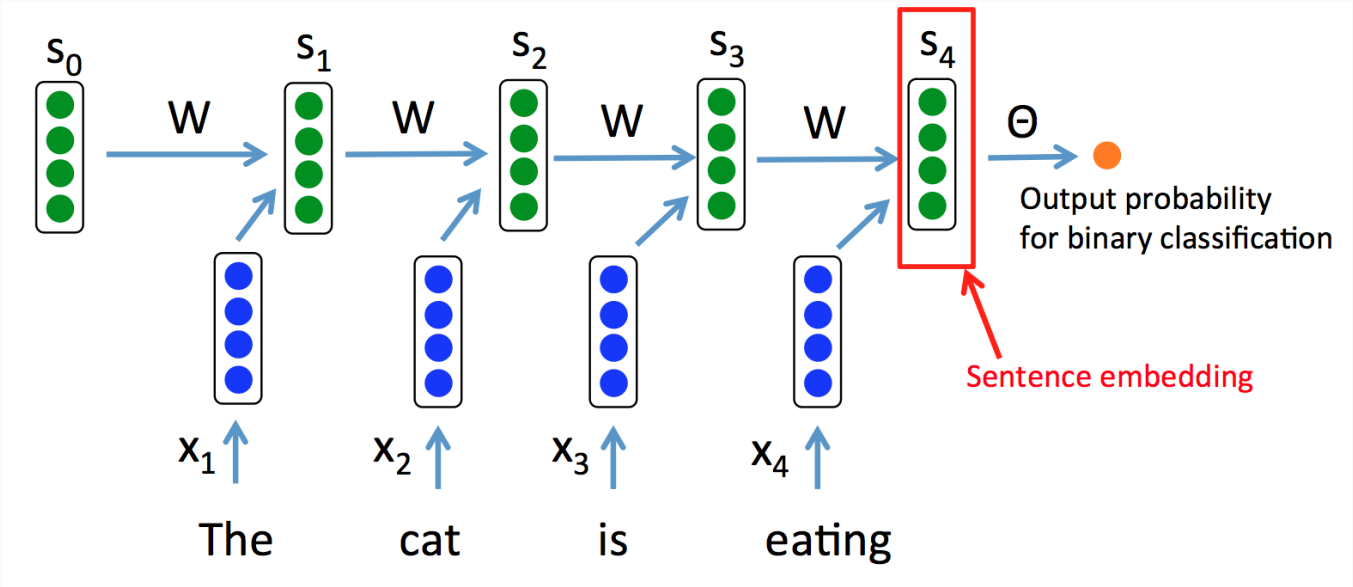
  Average loss over all sequences
- Solved by SGD/Adam

RNN: Text Classification

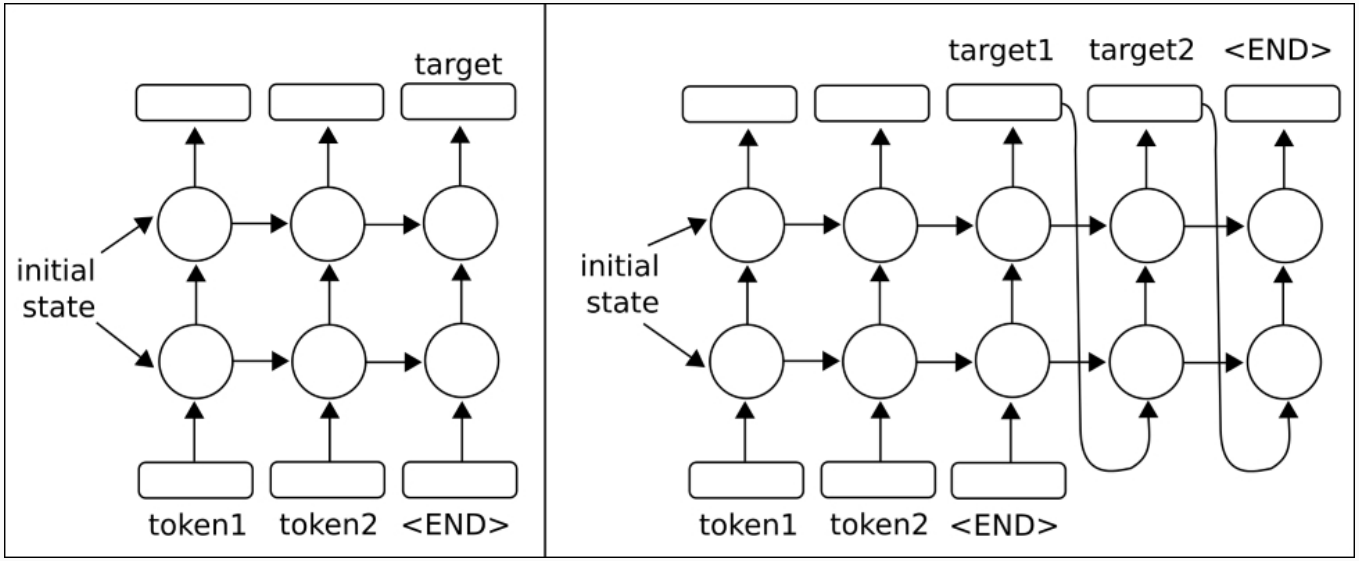
- Not necessary to output at each step
- Text Classification:

$$\text{sentence} \rightarrow \text{category}$$

  Output only at the final step
- Model: add a fully connected network to the final embedding



Multi-layer RNN



(Figure from `https://subscription.packtpub.com/book/big_data_and_business_intelligence`)

Problems of Classical RNN

- Hard to capture long-term dependencies
- Hard to solve (vanishing gradient problem)
- Solution:
  - LSTM (Long Short Term Memory networks)
  - GRU (Gated Recurrent Unit)
  - ...